

Exploring ISO_C_BINDING and type-bound procedures

Rohit Goswami · 2021-12-19 · fortran · f2py · ramblings

A closer look at the standard Fortran C compatibility layer by exploring objects and linkers

Background

[Derived types and their interoperability](#) have been covered previously in the context of usage from Python. However, much of the focus of the previous approach revolved around the `iso_c_binding` intrinsic module. A closer inspection of the functionality provided therein is the first step towards extending beyond the standards to support calling type bound procedures. This is an often overlooked aspect of the derived type usage pattern, in terms of interoperability.

Setup

We would like to understand the effect of the intrinsic `iso_c_binding` module. So, we will re-use at first, the basic cartesian type of the previous post, along with a **non-type-bound** procedure which is to be called from `C`.

```
! vec.f90
module vec
  implicit none
  integer, parameter :: dd = selected_real_kind(15,9)

  type :: cartesian
    real(kind=dd) :: x,y,z
  end type cartesian

contains

  subroutine unit_move(vec)
    type(cartesian), intent(inout) :: vec
    print*, "Modifying from Fortran"
    vec%x = vec%x + 1
    vec%y = vec%y + 1
    vec%z = vec%z + 1
  end subroutine unit_move

end module vec
```

There isn't a whole lot going on, except that the `iso_c_binding` labels have been dropped, and correspondingly, instead of `c_double` we now have an approximate `kind` mapping.

The associated `C` driver is exactly the same, though in this particular situation the function signature is optional.

```

/* vecfc.c */
#include<stdlib.h>
#include<stdio.h>
#include<vecfc.h>

void* unit_move(cartesian *word);

int main(int argc, char* argv[argc+1]) {
    puts("Initializing the struct");
    cartesian a={3.0, 2.5, 6.2};
    printf("%f %f %f",a.x,a.y,a.z);
    puts("\nFortran function with derived type from C:");
    unit_move(&a);
    puts("\nReturned from Fortran");
    printf("%f %f %f",a.x,a.y,a.z);
    return EXIT_SUCCESS;
}

```

To complete the translation unit, our header contains the `struct` definition.

```

#ifndef VECFC_H
#define VECFC_H

typedef struct {
    double x,y,z;
} cartesian;

#endif /* VECFC_H */

```

So as to not detract from the main focus of the post, instead of using `meson` or another build system, we will compile everything by hand.

Compilation

We will attempt to follow along the same logical process as in the `bind(c)` situation:

1. Compile and assemble the Fortran module
2. Compile, assemble and link the C driver with the module

A very literal attempt at satisfying the two step process above is perhaps as simple as:

```

gfortran -c vec.f90 # generates vec.o
gcc vec.o vecfc.c -I./ -lgfortran -o gf_vec
/usr/bin/ld: /tmp/cc00dInK.o: in function `main':
vecfc.c:(.text+0x9a): undefined reference to `unit_move'
collect2: error: ld returned 1 exit status

```

Naturally, the linker will fail at this point. Recall that we can check which symbols are actually part of `vec.o`.

```

nm vec.o

                 U _gfortran_st_write
                 U _gfortran_st_write_done
                 U _gfortran_transfer_character_write
                 U _GLOBAL_OFFSET_TABLE_
                 U __stack_chk_fail
0000000000000000 T __vec_MOD__copy_vec_Cartesian
0000000000000000 B __vec_MOD__def_init_vec_Cartesian
000000000000002c T __vec_MOD_unit_move
0000000000000000 D __vec_MOD__vtab_vec_Cartesian

```

Where the first few undefined functions are to be resolved by `-lgfortran` directive to the linker [^fn:1]. Note that the function we care to call is actually called `__vec_MOD_unit_move`. We can also check the symbols required by our program.

```
gcc -c vecfc.c -I./ -o gf_vec.o
nm -u gf_vec.o
                 U _GLOBAL_OFFSET_TABLE_
                 U printf
                 U puts
                 U __stack_chk_fail
                 U unit_move
```

So our problem is essentially one of renaming to the right symbol.

Symbol renaming

A first approximation towards a solution is then evident, we will forcibly rename the symbol in our Fortran code, thus allowing us to link and call the function. `objcopy` is fantastic for this.

```
gfortran -c vec.f90 # get vec.o
# Rename symbol
objcopy --redefine-sym=__vec_MOD_unit_move=unit_move vec.o
# Compile and link in one shot
gcc vec.o vecfc.c -I./ -lgfortran -o gf_vec
# Profit
./gf_vec
Initializing the struct
3.000000 2.500000 6.200000
Fortran function with derived type from C:
  Modifying the derived type now!

Returned from Fortran
4.000000 3.500000 7.200000
```

Indeed, apart from this very satisfying result, we can verify that our symbols are not undefined as well.

```
nm -u gf_vec
# nothing but library functions
```

Switching compilers

There are a number of caveats with the approach described so far, but perhaps one of the more striking ones has to do with changing the compiler. Consider the symbols generated by the Intel compiler.

```
ifort -c vec.f90
nm vec.f90
                 U for_write_seq_lis
0000000000000000 r __STRLITPACK_0
0000000000000000 r __STRLITPACK_1.0.2
0000000000000000 T vec._
0000000000000010 T vec_mp_unit_move_
```

As Fortran remains one of the few languages to have a rich and varied set of compilers with varying levels of support and standardisation, it would be rather a tall order to keep track of the symbol mangling approaches used by every compiler. Indeed, problematically, it is not just that the symbols mangled differently, the code generated is quantitatively different as well.

```

ifort vec.o vecfc.o -lc
ld: vecfc.o: in function `main':
vecfc.c:(.text+0x0): multiple definition of `main';
/opt/intel/oneapi/compiler/2021.2.0/linux/bin/intel64/../../compiler/lib/intel64_lin/for_main.o:for_main.c:
(.text+0x0): first defined here
ld: cannot find -lirng
ifort -c vec.f90 -fPIE
gcc vec.o vecfc.c -I./ -o gf_vec
/usr/bin/ld: vec.o: in function `unit_move':
vec.f90:(.text+0x55): undefined reference to `for_write_seq_lis'
collect2: error: ld returned 1 exit status

```

In any case, we recall from the `info` pages of `gfortran` that:

Note that just because the names match does not mean that the interface implemented by GNU Fortran for an external name matches the interface implemented by some other language for that same name. That is, getting code produced by GNU Fortran to link to code produced by some other compiler using this or any other method can be only a small part of the overall solution—getting the code generated by both compilers to agree on issues other than naming can require significant effort, and, unlike naming disagreements, linkers normally cannot detect disagreements in these other areas.

Simplifying labels

The first functionality of the `iso_c_binding` is actually rather easy to implement from a vendor perspective, but vastly simplifying for the end-user, the ability to provide a single binding label. Recall the `bind(c)` variant of the [previous post](#).

```

! vec_bind.f90
module vec
  use, intrinsic :: iso_c_binding
  implicit none

  type, bind(c) :: cartesian
    real(c_double) :: x,y,z
  end type cartesian

  contains

  subroutine unit_move(array) bind(c)
    type(cartesian), intent(inout) :: array
    print*, "Modifying the derived type now!"
    array%x=array%x+1
    array%y=array%y+1
    array%z=array%z+1
  end subroutine unit_move

end module vec

```

Which generates the following symbols.

```

gfortran -o vec.o -c vec_bind.f90
nm vec.o

                 U _gfortran_st_write
                 U _gfortran_st_write_done
                 U _gfortran_transfer_character_write
                 U _GLOBAL_OFFSET_TABLE_
                 U __stack_chk_fail
0000000000000002c T unit_move
0000000000000000 T __vec_MOD__copy_vec_Cartesian
0000000000000000 B __vec_MOD__def_init_vec_Cartesian
0000000000000000 D __vec_MOD__vtab_vec_Cartesian

```

This true across compilers as well, all without any invocations of `obj copy` and other approaches.

```

ifort -c vec.f90 -o vec.o
nm vec.o

                 U for_write_seq_lis
0000000000000000 r __STRLITPACK_1
0000000000000000 r __STRLITPACK_2.0.2
0000000000000010 T unit_move
0000000000000000 T vec._

```

Type-bound procedures

We would like to extend the discussion to beyond where the light of the standard reaches, that is to consider calling type-bound procedures, which are not supported by the standard. We will begin by a suitable modification of our code. In an ideal world, we would simply annotate the type-bound procedure.

```

! vec_typeb.f90
module vec
  use, intrinsic :: iso_c_binding
  implicit none

  type, bind(c) :: cartesian
    real(c_double) :: x,y,z
    contains
      procedure, pass(self) :: unitmv
  end type cartesian

  contains

  subroutine unit_move(self) bind(c)
    class(cartesian), intent(in) :: self
    print*, "Modifying the derived type now!"
    self%x=self%x+1
    self%y=self%y+1
    self%z=self%z+1
  end subroutine unit_move

end module vec

```

However, this understandably does not go very well.

```

ifort -c vec_typeb.f90
vec_typeb.f90(7): error #8575: A derived type with the BIND attribute shall not have a type bound procedure
part.
    contains
-----^
vec_typeb.f90(14): error #8224: A derived type used with the CLASS keyword shall not have the BIND attribute
or SEQUENCE property.    [CARTESIAN]
    class(cartesian), intent(in) :: self
gfortran -c vec.f90
vec.f90:7:13:

    7 |         contains
      |         1
Error: Derived-type 'cartesian' with BIND(C) must not have a CONTAINS section at (1)

```

Removing the attribute, from both the type and the subroutine and rearranging slightly, we get the following.

```

module vec
  use, intrinsic :: iso_c_binding
  implicit none

  type :: cartesian
    real(kind=8) :: x,y,z
    contains
      procedure, pass(self) :: unitmv
  end type cartesian

  contains

  subroutine unitmv(self)
    class(cartesian), intent(inout) :: self
    print*, "Modifying the derived type now!"
    self%x=self%x+1.0
    self%y=self%y+1.0
    self%z=self%z+1.0
  end subroutine unitmv

end module vec

```

The `class` attribute is required here instead of `type` since,

```

# ifort
error #8264: The passed-object dummy argument must be a polymorphic dummy data object if the type being
defined is extensible.

```

In any case, we are now in a position to inspect the generated object.

```

gfortran -c vec_typeb.f90
nm vec_typeb.o
                 U _gfortran_st_write
                 U _gfortran_st_write_done
                 U _gfortran_transfer_character_write
                 U _GLOBAL_OFFSET_TABLE_
                 U __stack_chk_fail
0000000000000000 T __vec_MOD__copy_vec_Cartesian
0000000000000000 B __vec_MOD__def_init_vec_Cartesian
000000000000002c T __vec_MOD__unitmv
0000000000000000 D __vec_MOD__vtab_vec_Cartesian

```

Which does not seem to be very different at all. We will make an attempt to directly modify the symbol-table as before.

```
objcopy --redefine-sym=__vec_MOD_unitmv=unit_move vec_typeb.o
```

An attempt to call this is bound for failure, a segmentation fault to be exact. In order to be able to call the type bound procedure then, we can begin by writing a wrapper subroutine.

```
subroutine unit_move(cartobj)
  type(cartesian), intent(inout) :: cartobj
  call cartobj%unitmv()
end subroutine
```

This does allow for the existing `C` interface to work.

```
gfortran -c vec_typeb.f90
objcopy --redefine-sym=__vec_MOD_unit_move=unit_move vec_typeb.o
gcc vec_typeb.o vecfc.c -I./ -o gf_vec -lgfortran
./gf_vec
Initializing the struct
3.000000 2.500000 6.200000
Fortran function with derived type from C:
  Modifying the derived type now!

Returned from Fortran
4.000000 3.500000 7.200000%
```

It is useful to at this point, that Fortran passes arguments by reference and not by value, so there are no additional copy related overheads incurred by this “wrapper” approach.

It was correctly pointed out on the [Fortran Discourse](#) that the pass-by-reference calling convention is not mandated by the standards, though [in practice](#) many compilers do pass by reference when the `VALUE` attribute is missing.

On the other hand, it does leave the programmer in a world bereft of the `iso_c_binding`, which means also that the mapping of types and precisions become rather fluid.

Conclusions

This brief interlude on derived types and functions is more relevant in the context of automated binding generators like `f2py` Peterson (2009).

Perhaps a more formal approach to wrapper generation is the one elaborated upon in great detail in the literature Gray, Roberts, and Evans (1999) which details the concept of a logical interface and a physical interface.

A similar but more pragmatic approach is the opaque pointer method used for derived types with pointers in Pletzer et al. (2008) and forms the basis for the implementation in `f90wrap` Kermode (2020).

The (ab)use of type-bound procedures in the context of these binding generation methodologies is to follow at some point since none of them make any explicit mention of the same. Given that type-bound procedures were not introduced before 2003 Reid (2003), it is not surprising they have been overlooked, however their usage is foundational for long-lasting, sustainable Fortran code.

Reading list

Fortran 2003 interoperability and the scientific codes that lean on it, collected as a citation list: [5 references](#). The collection exports to BibTeX or any CSL style, so it can go straight into a bibliography.

References

Gray, M. G., R. M. Roberts, and T. M. Evans. 1999. "Shadow-Object Interface Between Fortran 95 and C++." *Computing in Science Engineering* 1 (2): 63--70. <<https://doi.org/10.1109/5992.753048>>.

Kermode, James R. 2020. "F90wrap: An Automated Tool for Constructing Deep Python Interfaces to Modern Fortran Codes." *Journal of Physics: Condensed Matter* 32 (30): 305901. <<https://doi.org/10.1088/1361-648X/ab82d2>>.

Peterson, Pearu. 2009. "F2PY: A Tool for Connecting Fortran and Python Programs." *International Journal of Computational Science and Engineering* 4 (4): 296. <<https://doi.org/10.1504/IJCSE.2009.029165>>.

Pletzer, Alexander, Douglas McCune, Stefan Muszala, Srinath Vadlamani, and Scott Kruger. 2008. "Exposing Fortran Derived Types to C and Other Languages." *Computing in Science Engineering* 10 (4): 86--92. <<https://doi.org/10.1109/MCSE.2008.94>>.

Reid, John. 2003. "The New Features of Fortran 2003," 38. <<https://wg5-fortran.org/N1551-N1600/N1579.pdf>>.

interpreting the remaining symbols